

소켓 옵션

소켓 옵션 함수

지금까지 학습한 TCP와 UDP 프로그램에서 소켓은 기본 속성으로 생성하여 별다른 변경 없이 사용해 왔다. 그러나 소켓에는 송수신 버퍼 크기의 조정, 소켓주소 재할당 등과 같은 특별한 속성을 설정하여 해당 기능을 이용할 수도 있다. 어떤 속성들이 지원되는지 알아보기 전에 현재 소켓의 속성을 읽어오거나 변경하는 함수부터 살펴보자.

- 소켓 옵션 읽기/변경하기 함수

```
#include <sys/socket.h>
```

```
int getsockopt(int sockfd, int level, int optname, void *optval, socklen_t *optlen);
```

```
int setsockopt(int sockfd, int level, int optname, const void *optval, socklen_t optlen);
```

첫번째 인자 sockfd는 소켓 디스크립터를,
두번째 인자 level은 지정할 옵션의 프로토콜 레벨을,
세번째 인자 optname은 옵션 이름을,
네번째 인자 optval은 옵션별 필요한 값을,
마지막 인자는 optval의 자료형 길이를 읽어오거나 지정한다.
이 함수는 성공하면 0을 실패할 때는 -1을 반환한다.

프로토콜 별 소켓 옵션

■ 프로토콜과 소켓 관련 옵션

프로토콜	옵션 종류	변경 가능 여부
SOL_SOCKET	SO_SNDBUF, SO_RCVBUF, SO_REUSEADDR, SO_KEEPALIVE, SO_BROADCAST, SO_DONTROUTE, SO_OOBINLINE	O
	SO_ERROR, SO_TYPE	X
IPROTO_TCP	TCP_KEEPALIVE, TCP_NODELAY, TCP_MAXSEG	O

소켓 옵션의 종류

■ SOL_SOCKET 프로토콜 레벨

옵션 종류	설명	옵션값 유형
SO_SNDBUF	데이터 전송을 위한 버퍼공간 명시	int
SO_RCVBUF	데이터 수신을 위한 버퍼공간 명시	int
SO_REUSEADDR	주소의 재사용 (bind) 허용	int
SO_KEEPALIVE	TCP의 KEEPALIVE 모드 동작	int
SO_BROADCAST	UDP의 브로드캐스트 허용	int
SO_DONTROUTE	라우팅 없이 직접 네트워크 인터페이스로 전송	int
SO_OOBINLINE	OOB 데이터를 일반 데이터와 동일하게 취급	int
SO_LINGER	소켓 해제시 데이터 처리에 대한 규칙	struct linger

소켓 옵션의 종류

■ IPPROTO_TCP 프로토콜 레벨

옵션 종류	설명	옵션값 유형
TCP_NODELAY	Nagle 알고리즘 비활성화 유무	int
TCP_MAXSEG	TCP의 최대 세그먼트 크기 지정	int
TCP_SYNCNT	SYN 패킷을 재시도할 수 있는 최대 횟수	int
TCP_LINGER2	FIN_WAIT2의 타임 아웃	int
TCP_KEEPCNT	SO_KEEPALIVE 설정시 이 횟수만큼 응답이 없는 경우 연결 해제	int
TCP_KEEPINTVL	KEEPALIVE 패킷 전송 시간 간격 (초)	int

소켓 옵션 : SO_REUSEADDR

앞서 소켓의 옵션에는 다양한 종류가 있음을 살펴보았다. 이제 그 옵션의 의미와 활용 방법을 생각해보자. 여기에서는 가장 많이 사용되는 대표적인 옵션들에 대해서만 살펴본다. 소개되지 않는 옵션들에 대해서도 온라인 매뉴얼이나 인터넷 검색을 통해 확인해보기 바란다.

■ SO_REUSEADDR 옵션

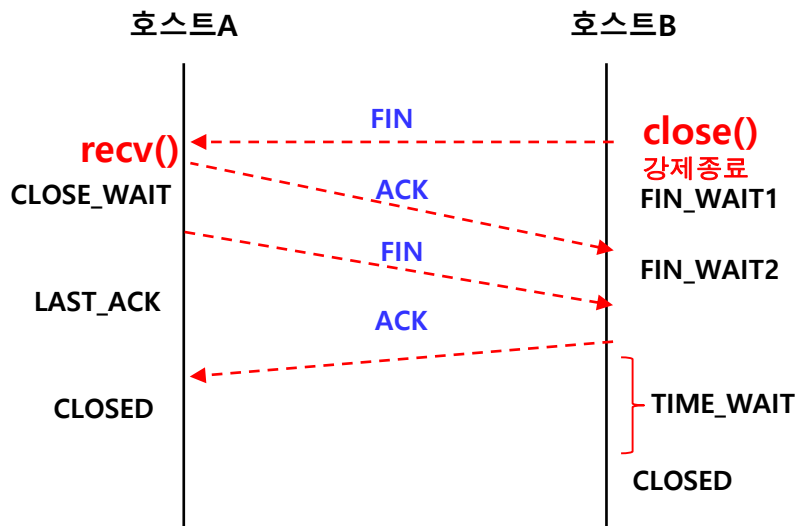
TCP 서버 프로그램을 수행하다가 클라이언트에서 먼저 종료하는 경우에는 큰 문제가 없지만 만약 서버가 예기치 못한 상태로 종료되는 경우 서버를 다시 실행하면 다음과 같은 오류 메시지가 나오면서 실행되지 않는다.

“ *bind : Address already in use* ”

이와 같은 오류메시지가 출력되는 이유는 `bind` 함수 실행 시 새로운 소켓에 설정하려는 주소가 이미 다른 소켓에서 사용 중이기 때문이다. 그러나 2~3분 후에 다시 서버를 실행해보면 `server` 프로그램이 잘 실행된다. 이런 상황은 왜 발생하는지부터 알아보자. (continue)

소켓 옵션 : SO_REUSEADDR

일반적으로 서버는 무한 루프 구조의 프로그램으로 작성된다. 그런데 서버를 강제로 종료시키거나 예외 상황이 일어나 종료하는 경우 서버 측에 연결되어있던 클라이언트와의 서비스를 모두 강제 종료시킨다. 이때 서버 쪽의 `LISTEN` 상태에서 기다리고 있던 소켓은 “`TIME_WAIT`” 상태로 전이가 되면서 1분 후 종료가 된다. 따라서 이 시간 동안 서버에서는 `bind` 할 수 없게 된다. 이것을 그림으로 표현하면 다음과 같다.



서버의 경우 다시 실행하게 되면 `bind` 함수에서 동일한 서버의 IP 주소와 서비스 포트 번호로 소켓주소를 지정하므로 오류가 발생한다. 이런 오류가 일어나지 않도록 할 때 사용하는 옵션이 바로 `SO_REUSEADDR` 이다.

소켓 옵션 : SO_SNDBUF, SO_RCVBUF

- SO_SNDBUF와 SO_RCVBUF 옵션은 TCP나 UDP 프로토콜에서 송수신 버퍼의 크기를 확인하거나 변경할 때 사용하는 옵션으로 옵션 값은 바이트 단위의 버퍼 크기를 의미한다.
- 리눅스 커널에서는 setsockopt로 설정 한 값의 2배를 송수신 버퍼의 크기로 할당하는데, 이는 부가 정보를 저장하기 위한 영역을 확보하기 때문이다. 송수신 버퍼의 크기는 다음과 같은 파일에서 확인할 수도 있다.

버퍼	기본 크기	설정 가능한 최대 크기
송신 버퍼	/proc/sys/net/core/wmem_default	/proc/sys/net/core/wmem_default
수신 버퍼	/proc/sys/net/core/rmem_default	/proc/sys/net/core/rmem_max

주의할 점은 프로그램에서 송수신 버퍼의 크기를 조정하는 것은 자유롭게 할 수 있지만 실제 적용 여부는 시스템의 자원 현황에 따라 다를 수 있다. 최신의 리눅스 커널에서는 TCP auto tuning 기능이 추가되면서 시스템의 성능을 위해 버퍼의 크기를 동적으로 관리할 수 있으므로 요청한 크기와 정확히 일치하지 않을 수도 있다

소켓 옵션 : SO_LINGER

- SO_LINGER는 TCP에서만 사용하는 옵션으로 연결 해제시 송신 버퍼의 데이터가 완전히 전송되었는지 확인하기 위해 사용하거나 TIME_WAIT 상태를 만들지 않기 위해 사용할 수 있다. SO_LINGER 옵션에 사용되는 구조체 자료형은 다음과 같다.

```
struct linger {  
    int l_onoff;  
    int l_linger;  
}
```

- l_onoff == 0 인 경우
l_linger와 무관함.소켓의 기본설정으로 소켓버퍼에 남아 있는 모든 데이터를 전송하고 종료한다.
- l_onoff > 0 이고 l_linger == 0 인 경우
소켓버퍼에 아직 남아있는 데이터는 모두 버리고 즉시 종료한다. 상대방에게는 이 데이터 대신 리셋을 위한 RST 패킷을 보낸다. 따라서 TIME_WAIT 가 발생하지 않는다. 이를 'abortive 종료' 라고 부른다.
- l_onoff > 0 이고 l_linger > 0 인 경우
버퍼에 남아있는 데이터를 모두 전송하고 종료한다. 이때 close() 함수에서는 l_linger에 지정된 시간만큼 블록상태에서 전송을 마치기를 기다린다. 시간 내에 전송이 완료되면 'TIME_WAIT' 상태로 천이되지만 전송이 완료되지 않은 경우에는 강제로 소켓이 해제되고 송신 버퍼의 내용이 모두 삭제된다. 이와 같이 전송할 수 있도록 유예시간을 두고 종료하는 것을 'graceful 종료' 라고 한다.

소켓 옵션 : TCP_NODELAY

- TCP_NODELAY 옵션은 TCP 프로토콜에 적용되어 있는 Nagle 알고리즘을 비활성화 할 수 있도록 설정하는 옵션이다. 여기서 Nagle 알고리즘이란 네트워크상에서 돌아다니는 패킷들이 너무 빈번하게 자주 송수신되는 것을 막기 위해 고안된 알고리즘으로 기존에 전송한 패킷이 있을 경우 그 패킷에 대한 ACK를 받아야만 다음 전송을 진행하는 알고리즘이다.
- Nagle 알고리즘이 적용되면, 커널에서는 ACK가 오기를 기다렸다가 도착하면, 그 동안 쌓여있던 패킷들을 한꺼번에 보내기 때문에 대역폭이 낮은 WAN에서는 전송 횟수를 줄여주는 효과가 있다. 네트워크의 효율적 사용을 위해 TCP소켓은 기본적으로 Nagle 알고리즘이 적용된다.
- 하지만 경우에 따라서는 네트워크 의 전송량이나 부하보다는 빠른 응답속도가 중요한 경우에는 Nagle 알고리즘을 비활성화 함으로써 해결할 수 있다. 이런 상황은 주로 LAN 환경에서 대화형의 통신 프로그램을 수행할 때 필요하다. 주로 LAN 환경에서 응답시간을 요하는 프로그램이나 WAN 환경에서 실시간 게임이나 온라인 채팅 프로그램의 경우 이 옵션을 설정하는 경우가 많다

사용 예>

```
setsockopt(sockfd, IPPROTO_TCP, TCP_NODELAY, &opval, sizeof(opval));
```

〈실습〉

- 다음과 같은 조건의 소켓 프로그램을 작성해봅시다.

〈서버 : msg_server〉

- 2개의 UDP 소켓(포트번호 60000, 60001)을 이용하여 클라이언트가 보내는 메시지를 수신
- 서버에 수신된 메시지를 msg_collect 프로세스(또는 쓰레드)로 전송
- msg_collect 프로세스(또는 쓰레드)에서 날짜(strftime()) 라이브러리 함수 이용)와 클라이언트 IP 주소를 포함하여 msg.log 파일에 저장

파일 저장 형식 예) 2002년 09월 30일 10:34:36 10.0.2.15 클라이언트 메시지

〈클라이언트 : msg_client〉

- 클라이언트는 우분투 리눅스와 라즈비안에서 모두 실행
 - 서버이름과 포트번호(60000또는 60001)와 전송 메시지를 실행 인자로 전달하여 서버로 전송
- 실행 예) ./msgclient localhost 60000 hello1